

Правительство Российской Федерации

**Федеральное государственное автономное образовательное учреждение
высшего профессионального образования
"Национальный исследовательский университет
"Высшая школа экономики"**

Московский институт электроники и математики Национального
исследовательского университета "Высшая школа экономики"

Департамент прикладной математики

ОТЧЕТ

По лабораторной работе № 7

По курсу «Алгоритмизация и программирование»

ФИО студента	Номер группы	Дата
Индюченко Никита Андреевич	БПМ211	26.01.2022

Москва – 2022г.

ЗАДАНИЕ (вариант №12)

ОБРАБОТКА СТРОК

Написать функцию обработки строки и программу, тестирующую данную функцию. В программе должен быть предусмотрен вывод исходной строки, которая при выделении слов не должна измениться.

Текст задания Вашего варианта

12	Дана строка, содержащая от 1 до 30 слов, в каждом из которых от 1 до 10 латинских букв и/или цифр; между соседними словами – запятая, за последним словом – точка. Напечатать те слова, перед которыми в последовательности находятся только меньшие (по алфавиту) слова, а за ними – только большие.
----	---

РЕШЕНИЕ

Код программы с комментариями

```
#define _CRT_SECURE_NO_WARNINGS // чтобы могли пользоваться строковыми функциями (убрать ограничения)
```

```
#include <stdio.h>
```

```
#include <string.h> // библиотека, в которой str функции
```

```
#include <stdlib.h>
```

```
void print_string(char* _str); // функция, которая выводит слова, которые удовлетворяют условию задачи
```

```
void print_original_str(char* _str); // функция, которая выводит исходную строку
```

```
void scanf_string(char* _str); // функция, которая заполняет строку
```

```
int main(void) {
```

```
    char* s; // объявление строки
```

```
    s = (void*)malloc(330 * sizeof(char)); // выделение памяти
```

```
    if (s == NULL)
```

```
    {
```

```
        printf("Memory is not allocated\n");
```

```
        return 1;
```

```
    }
```

```
    printf("Enter string:\n");
```

```
    //scanf_string(s); // считывание слов в строку
```

```
    fgets(s, 330, stdin);
```

```
    print_string(s); // вывод строки, в которой хранятся слова по условию задачи
```

```
    printf("Original string:\n");
```

```
    //print_original_str(s); // вывод исходной строки
```

```
    puts(s);
```

```
    free(s);
```

```
}
```

```
void scanf_string(char* _str)
```

```
{
```

```
    char tmp;
```

```
    int i = 0;
```

```
    do
```

```
    {
```

```
        scanf_s("%c", &tmp);
```

```
        _str[i] = tmp;
```

```
        i++;
```

```
    } while ((tmp != '.') || (tmp != '\n'));
```

```
}
```

```

void print_string(char* _str)
{
    char* str_1;
    str_1 = (void*)malloc(330 * sizeof(char));
    if (str_1 == NULL)
    {
        printf("Memory is not allocated in function <print_string>\n");
    }
    else
    {
        char str_2[338] = "String:\n"; // строка, в которой будут храниться все
        подходящие слова по условию
        str_1 = strcpy(str_1, _str); // strcpy - копирует строку str в str_1 (включая
        '\0'), возвращает указатель на str_1 (char*)
        int str_in_1_word = 0; // счётчик кол-во слов
        for (int i = 1; i < strlen(_str); i++)
        {
            if (((_str[i] == ',') || (_str[i] == ' ') || (_str[i] == '.')) && (_str[i -
1] != ',') && (_str[i - 1] != ' ') && (_str[i - 1] != '.'))
            {
                str_in_1_word++;
            }
            if (_str[i] == '.')
            {
                break;
            }
        }
        if (str_in_1_word == 0)
        {
            printf("The string consists of zero word\n");
        }
        else if (str_in_1_word == 1) // проверка на слова
        {
            printf("The string consists of one word\n");
        }
        else if (str_in_1_word == 2)
        {
            printf("the string consists of two words\n");
        }
        else if (str_in_1_word > 29)
        {
            printf("More than 30 words\n");
        }
        else // если 3 и более
        {
            char* word_1, * word_2, * word_3; // объявление строк и выделение памяти
            word_1 = (void*)malloc(10 * sizeof(char));
            word_2 = (void*)malloc(10 * sizeof(char));
            word_3 = (void*)malloc(10 * sizeof(char));
            if ((word_1 == NULL) || (word_2 == NULL) || (word_3 == NULL))
            {
                printf("Memory is not allocated in function <print_string>\n");
            }
            else
            {
                word_1 = strtok(str_1, " ,."); // выделение слова из строки str_1 по
                токенам при помощи strtok - типа char* возвращает указатель на первый элемент не входящий
                в список токенов
                word_2 = strtok(NULL, " ,."); // так как выделили первое слово, то второе
                начинается с указателя который запомнил в прошлом вызове (пишем NULL)
                word_3 = strtok(NULL, " ,."); // аналогично берём 3-ье слово
                int i = 0; // счётчик проходов в цикле
                while (word_3 != NULL)
                {
                    i++;
                }
            }
        }
    }
}

```

```

        if (i + 1 == str_in_1_word) break; // чтобы не учитывал случай, когда
        второе слово является последним в строке.
        if ((strcmp(word_2, word_1) > 0) && (strcmp(word_3, word_2) > 0)) //
        strcmp сравнивает строки, возвращает типа int. если первая строка больше 2-ой, то >0,
        обратное - <0. Равны - 0
    {
        strcat(str_2, word_2); // вставка в str_2 слово word_2. Типа char*
        strcat(str_2, " ");
    }
    word_1 = strcpy(word_1, word_2);
    word_2 = strcpy(word_2, word_3);
    word_3 = strtok(NULL, " ,.");
}
puts(str_2);
// аналог вывода строки, написанная в библиотеке "string.h". Тип int
free(word_1);
word_2 = NULL;
free(word_2);
word_3 = NULL;
free(word_3);
}
}
str_1 = NULL;
free(str_1);
}
}

```

```

void print_original_str(char* _str)
{
    for (int i = 0; i < strlen(_str); i++) // функция strlen возвращает кол-во символов в
    строке (int).
    {
        printf("%c", _str[i]);
        if (_str[i] == '.') break;
    }
}

```

ТЕСТЫ

Тест № 1

Если 1 слово

Результаты теста 1

```

Enter string:
abc.
The string consists of one word
Original string:
abc.

```

Тест № 2

Если 2 слова

Результаты теста 2

```

Enter string:
abc,cccccd.
the string consists of two words
Original string:
abc,cccccd.

```

Тест № 3

Пример с двумя словами, которые подходят и являются соседними

Результаты теста 3

```
Enter string:
abc,cde,fff,zzz,aaa.
String:
cde fff
Original string:
abc,cde,fff,zzz,aaa.
```

Тест № 4

Если будет более 30 слов

Результаты теста 4

```
Enter string:
c1,c2,c3,c4,c5,c6,c7,c8,c9,c10,d1,d2,d3,d4,d5,d7,d8,d9,d10,f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,eda.
More than 30 words
Original string:
c1,c2,c3,c4,c5,c6,c7,c8,c9,c10,d1,d2,d3,d4,d5,d7,d8,d9,d10,f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,eda.
```

Тест № 5

Если все слова одинаковые

Результаты теста 5

```
Enter string:
aaa,aaa,aaa,aaa.
String:

Original string:
aaa,aaa,aaa,aaa.
```

Тест № 6

Проверка на строку из цифр

Результаты теста 6

```
Enter string:
1,2,3,4,5.
String:
2 3 4
Original string:
1,2,3,4,5.
```

Тест № 7

Проверка на несколько предложений, то есть чтобы не сравнивал слова после точки.

Результаты теста 7

```
Enter string:
abc,bbb,ccc,fff,zzz,aaa. abc,bbb,ccc,fff,zzz,aaa, abc,bbb,ccc,fff,zzz,aaa.
String:
bbb ccc fff
Original string:
abc,bbb,ccc,fff,zzz,aaa. abc,bbb,ccc,fff,zzz,aaa, abc,bbb,ccc,fff,zzz,aaa.
```

Тест № 8

Проверка на несколько знаков препинаний (,) или пробелов.

Результаты теста 8

```

Enter string:
a,,,,,      ffff.
the string consists of two words
Original string:
a,,,,,      ffff.

```

Тест № 9

Проверка на выделение памяти исходной строки, а также дополнительной динамической памяти в функции

Результаты теста 9

```

if ((word_1 == NULL) || (word_2 == NULL) || (word_3 == NULL) || (str_1 == NULL))
{
    printf("Memory is not allocated in function <print_string>\n");
}

```

Enter string:
abc,ddd,fdfdf,fdfdf.
Memory is not allocated in function <print_string>
Original string:
abc,ddd,fdfdf,fdfdf.

```

char s, // объявление строки
s = (void*)malloc(1000000000000000 * sizeof(char));

```

Memory is not allocated

Тест № 10

Если нет слов в строке, а только знаки препинания

Результаты теста 10

```

Enter string:
.....
The string consists of zero word
Original string:
.....

```

Тест № ...

Результаты теста ...